AISES Software Evaluation Report

Executive Summary

Metric	Value
Total Requirements	12
Implemented	12
Partially Implemented	0
Not Implemented	0
Overall Compliance	100.0 %

Requirement Evaluation

AC-001 : Numeric Input and Multi-digit Numbers

Digits 0 through 9 append correctly to the input display when pressed.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Buttons for digits 0 through 9 are correctly mapped to the digit method with their respective string values.

• The digit method handles replacing initial zero/new input flags as well as appending subsequent digits properly.

Issues

None

Recommendations

None

AC-002 : Numeric Input and Multi-digit Numbers

Only one decimal point is allowed per operand.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The decimal method explicitly checks if '.' is already present in the current display value before appending a new decimal point.

Issues

None

Recommendations

None

AC-003 : Numeric Input and Multi-digit Numbers

Multi-digit numbers are supported and form correctly when consecutive digits are entered.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The digit method correctly handles string concatenation of consecutive digit inputs when self.new_input is False and the current display is not '0'.

Issues

None

Recommendations

None

AC-004 : Basic Arithmetic Operations

Addition operation computes the sum of operands correctly.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The '+' button properly registers the addition operator via set_operator.

• The calculate method defines the addition operation lambda: self.first + second which correctly computes the sum of the operands.

• Operands are correctly parsed as floats before calculation.

Issues

None

Recommendations

None

AC-005 : Basic Arithmetic Operations

Subtraction operation computes the difference of operands correctly.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Subtraction is correctly mapped using self.first - second in the operations dictionary.

• Operands are properly parsed as floats.

• Result formatting correctly handles integer representations.

Issues

None

Recommendations

None

AC-006 : Basic Arithmetic Operations

Multiplication operation computes the product of operands correctly.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The multiplication operator '*' is mapped correctly to set_operator.

• The calculate method implements multiplication using 'self.first * second'.

Issues

None

Recommendations

None

AC-007 : Basic Arithmetic Operations

Division operation computes the quotient of operands correctly.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Division operation is correctly implemented using the '/' operator in the calculate method.

• Handles division by zero properly with a ZeroDivisionError exception handler.

• Maps the division button '/' cleanly to the set_operator function.

Issues

None

Recommendations

None

AC-008 : Clear State

Clear (C) button resets all calculator state and display values.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The 'C' button is correctly mapped to the clear method in button_data.

• The clear method resets the display value to '0', first operand to 0.0, operator to None, and new_input to True, fully resetting calculator state.

Issues

None

Recommendations

None

AC-009 : Delete Last Character

Delete (DEL) button removes the last entered character from the input display.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The DEL button is correctly mapped to the delete method.

• The delete method properly slices off the last character using current[:-1] when length is greater than 1.

• The delete method falls back to '0' and resets new_input when only one character remains.

Issues

None

Recommendations

None

AC-010 : Division by Zero Validation

Division by zero is detected before calculation and an error message is displayed instead of performing the calculation.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Division by zero is explicitly handled using a try-except block catching ZeroDivisionError.

• An error message ('Cannot divide by zero') is displayed to the user upon division by zero.

Issues

None

Recommendations

None

AC-011 : Evaluate Expression

Pressing '=' computes the result of the current expression.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The '=' button is successfully bound to the calculate method in button_data.

• The calculate method properly evaluates the current expression based on the operator and operands.

• Error handling for division by zero and unexpected exceptions is correctly implemented.

Issues

None

Recommendations

None

AC-012 : Display Formatted Numeric Results

Results are formatted correctly and presented clearly on the display field.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Results are formatted by converting integer values to integer strings and keeping floats as-is

• Display is updated properly via StringVar tied to the UI Entry widget

• Exceptions such as division by zero and generic errors are caught and cleanly displayed

Issues

None

Recommendations

None